

3-tier 학생 관리 CRUD 실습

Python 기초부터 읽는 애플리케이션 코드

Week 3-2 데이터베이스
1분반 9/18 · 2분반 9/17

03

3-tier
90분 실습

실습 목표

Python 기초

dict, tuple, 함수와 self를 실습 코드에서 읽습니다.

요청의 이동

등록 요청이 화면·서버·DB를 거치는 과정을 따라갑니다.

코드 설명

검증, SQL 바인딩, 커밋과 응답의 역할을 설명합니다.

90분 실습 순서

시간	활동
0-5분	환경 확인과 서버 실행
5-30분	Python 필수 문법과 예제 실행
30-45분	CRUD 실행과 HTTP 관찰
45-75분	등록 요청을 따라 애플리케이션 코드 읽기
75-85분	오류·롤백 확인과 코드 설명
85-90분	확인 문항 제출

논리적 3계층 구조

계층	구성 요소	역할
화면	브라우저	입력과 결과 표시
애플리케이션	Python 서버	입력 검증과 CRUD 처리
데이터	데이터 접근 모듈·SQLite	SQL 실행과 저장

SQLite 엔진은 Python 서버 프로세스 안에서 실행됩니다.

실행 준비

Python 3.10 이상, 외부 패키지와 C 컴파일러 설치 불필요

```
python3 server.py
```

```
http://127.0.0.1:8765/
```

Windows: `py -3 server.py`

포트 변경: `python3 server.py --port 8766`

Python을 어디서 실행할까

파일	언어	실행 위치
server.py · service.py	Python	서버
repository.py	Python과 SQL 문자열	서버와 SQLite
static/app.js	JavaScript	브라우저

Python 연습: python3 python_basics.py / Windows: py -3 python_basics.py

변수와 값의 타입

Python 기초 · 실제 애플리케이션 코드를 읽기 위한 준비

```
student_id = 2001
name = "O'Brien"
credits = 12
dept = None
print(type(credits).__name__) # int
```

=는 대입입니다. 숫자, 문자열, None을 구분합니다.
None은 값이 없음을 나타내며 문자열 "None"과 다릅니다.

문자열 숫자와 정수

Python 기초 · 실제 애플리케이션 코드를 읽기 위한 준비

```
print('12' + '1')           # 121
print(int('12') + 1)       # 13
print(f'학번 {student_id}: {credits}학점')
```

int는 문자열을 정수로 변환합니다.
f-string의 중괄호에는 변수의 값을 넣습니다.

들여쓰기와 조건문

Python 기초 · 실제 애플리케이션 코드를 읽기 위한 준비

```
if 0 <= credits <= 200:
    print('유효한 학점')
else:
    print('학점 범위 오류')
print('검사 종료')
```

콜론 뒤의 들여쓰기 줄이 해당 분기의 블록입니다.
마지막 print는 어느 분기가 실행되든 수행합니다.

비교 · 논리 연산과 None

Python 기초 · 실제 애플리케이션 코드를 읽기 위한 준비

```
if dept is None:  
    print('학과 미정')
```

```
if type(credits) is not int or not 0 <= credits <= 200:  
    print('정수가 아니거나 범위를 벗어났습니다.')
```

and는 모두 참, or는 하나 이상 참, not은 반전입니다.
None 검사는 is None으로 씁니다.

딕셔너리: 키로 값 읽기

Python 기초 · 실제 애플리케이션 코드를 읽기 위한 준비

```
body = {'id': 2001, 'name': "O'Brien", 'tot_cred': 12}  
print(body['name'])  
print(body.get('email', '없음'))  
body['tot_cred'] = 24
```

body['tot_cred']는 학점 값을 읽습니다.
get은 키가 없을 때 기본값을 돌려줍니다.

리스트와 튜플의 순서

Python 기초 · 실제 애플리케이션 코드를 읽기 위한 준비

```
rows = [{'id': 2001}, {'id': 2002}]
params = (24, 2001)
print(params[0], params[1])    # 24 2001
one = (2001,)                  # 원소 하나인 튜플
```

인덱스는 0부터 시작합니다. (2001)은 정수입니다.
UPDATE의 두 ?에는 (학점, 학번) 순서로 전달합니다.

반복문을 한 줄씩 읽기

Python 기초 · 실제 애플리케이션 코드를 읽기 위한 준비

```
ids = []  
for row in rows:  
    ids.append(row['id'])  
print(ids)  # [2001, 2002]
```

```
ids = [row['id'] for row in rows]
```

아래 한 줄은 위 반복문과 같은 리스트를 만듭니다.
처음에는 긴 반복문으로 풀어서 읽습니다.

함수와 return

Python 기초 · 실제 애플리케이션 코드를 읽기 위한 준비

```
def valid_credits(value, maximum=200):  
    return type(value) is int and 0 <= value <= maximum
```

```
print(valid_credits(12))          # True  
print(valid_credits('12'))       # False  
print(valid_credits(12, 10))     # False
```

def는 정의, 함수명(...)은 호출입니다.
return은 값을 돌려주며 print는 화면에 출력합니다.

import와 파일의 시작 지점

Python 기초 · 실제 애플리케이션 코드를 읽기 위한 준비

```
import json
from repository import Repository

if __name__ == '__main__':
    main()
```

import는 다른 모듈의 이름을 가져옵니다.
이 조건은 파일을 직접 실행했을 때만 main을 호출합니다.

클래스와 self

Python 기초 · 실제 애플리케이션 코드를 읽기 위한 준비

```
class Student:
    def __init__(self, name):
        self.name = name
    def greeting(self):
        return f'학생: {self.name}'
```

```
first = Student('김민지')
print(first.greeting())
```

`__init__`은 객체의 초기 상태를 설정합니다.
`first.greeting()`에서 `self`는 `first` 객체를 가리킵니다.

실습의 self.repository 읽기

Python 기초 · 실제 애플리케이션 코드를 읽기 위한 준비

```
class Service:
    def __init__(self, repository):
        self.repository = repository
```

```
# run()의 등록 처리에서
return self.repository.add((sid, name, dept, credits))
```

생성 시 받은 저장소를 객체에 기억합니다.
나중에 그 저장소의 add 메서드를 호출합니다.

예외 처리: try와 except

Python 기초 · 실제 애플리케이션 코드를 읽기 위한 준비

```
try:
    credits = int('열둘')
except ValueError:
    print('정수로 바꿀 수 없습니다.')
finally:
    print('변환 시도 종료')
```

raise는 오류를 발생시키고 except는 오류를 처리합니다.
서비스의 입력 오류는 서버에서 HTTP 400으로 바꿉니다.

with의 역할을 나누어 읽기

Python 기초 · 실제 애플리케이션 코드를 읽기 위한 준비

```
with closing(self.connect()) as db:
    with db:
        db.execute(sql, params)
```

실제 예제는 같은 구조를 한 줄로 씁니다.

```
with closing(self.connect()) as db, db:
    cursor = db.execute(sql, params)
```

with db는 열린 트랜잭션을 커밋하거나 롤백합니다.
closing은 마지막에 연결을 닫습니다.

JSON과 Python 값의 변환

Python 기초 · 실제 애플리케이션 코드를 읽기 위한 준비

```
body = {'dept_name': None, 'active': True}
wire = json.dumps(body)
print(wire)  # {"dept_name": null, "active": true}
again = json.loads(wire)
print(again['dept_name'] is None)  # True
```

Python의 None·True는 JSON에서 null·true입니다.
이제 body['tot_cred']와 params를 실제 코드에서 찾습니다.

CRUD와 HTTP 요청

작업	메서드	경로
조회	GET	/api/students
등록	POST	/api/students
학점 수정	PATCH	/api/students/2001
삭제	DELETE	/api/students/2001

브라우저는 SQL 대신 작업 경로와 입력값을 전송합니다.

등록과 이름 검색

등록

2001 / O'Brien / 컴퓨터공학 / 12를 입력합니다.

확인

HTTP 201, changes=1, 전체 학생 6명을 확인합니다.

검색

이름 O'Brien을 검색하고 한 행이 나오는지 확인합니다.

검색은 부분 일치가 아닌 정확한 이름 일치입니다.

수정과 삭제

수정

학번 2001의 학점을 24로 바꿉니다.

삭제

학번 2001을 삭제하고 검색 결과 0행을 확인합니다.

없는 학생

같은 학번을 다시 삭제하면 HTTP 404가 반환됩니다.

코드 파일과 책임

파일	핵심 함수	담당 역할
static/app.js	request · list · render	입력, HTTP, 화면 갱신
server.py	dispatch · send	경로 처리와 JSON 응답
service.py	Service.run	입력 검증과 작업 선택
repository.py	add · execute	고정 SQL, 바인딩, 저장

등록 요청의 전체 흐름

위치	전달하는 데이터
브라우저 request	POST /api/students와 JSON 본문
서버 dispatch	메서드, 학번, Python dict
서비스 run	검증한 (학번, 이름, 학과, 학점)
저장소 add · execute	INSERT 템플릿과 매개변수
서버 send / 브라우저 render	HTTP 201, 후속 GET과 목록 표시

객체를 연결하는 시작 코드

server.py · make_server()

```
repository = Repository(db_path)
repository.initialize()
server = ThreadingHTTPServer(('127.0.0.1', port), Handler)
server.service = Service(repository)
return server
```

서비스에 저장소 객체를 전달합니다.
HTTP 처리와 SQL 실행을 파일별로 나누어 읽습니다.

폼 제출과 입력값 변환

static/app.js · add-form 이벤트

```
$( 'add-form' ).onsubmit = (event) => {  
  event.preventDefault();  
  action(async () => {  
    const form = new FormData(event.target);
```

preventDefault는 기본 폼 제출을 막습니다.
FormData는 입력창의 name 속성으로 값을 읽습니다.

등록 요청의 본문

static/app.js · POST 호출 · 줄바꿈 조정

```
await request('POST', '/api/students', {
  id: Number(form.get('id')),
  name: form.get('name'),
  dept_name: form.get('dept_name') || null,
  tot_cred: Number(form.get('tot_cred'))
});
await list(false);
```

숫자와 null로 변환한 값을 서버에 보냅니다.
등록 성공 뒤에 별도 GET으로 목록을 갱신합니다.

JSON 직렬화와 비동기 요청

static/app.js · request() 발췌

```
const options = {method, headers: {}};
if (body !== undefined) {
  options.headers['Content-Type'] = 'application/json';
  options.body = JSON.stringify(body);
}
// 관찰창 표시 코드는 생략
const response = await fetch(path, options);
const data = await response.json();
```

객체를 JSON 문자열로 보내고 응답 JSON을 읽습니다.
두 프로세스가 같은 객체를 공유하는 것은 아닙니다.

HTTP 메서드와 경로 처리

server.py · Handler

```
do_GET = do_POST = do_PATCH = do_DELETE = dispatch
```

```
# dispatch() 안의 경로 처리
```

```
route = re.fullmatch(r'/api/students(?:/([0-9]+))?', url.path)
```

```
# 경로 오류 처리는 생략
```

```
sid = int(route[1]) if route[1] else None
```

GET·POST·PATCH·DELETE가 dispatch를 사용합니다.

목록 경로는 sid=None, /2001 경로는 sid=2001입니다.

요청 JSON을 Python 값으로 읽기

server.py · dispatch() 발췌

```
length = int(self.headers.get('Content-Length', '0'))
if not 0 < length <= 8192:
    raise InputError('요청 본문은 1~8192바이트여야 합니다.')
body = json.loads(self.rfile.read(length))
```

JSON의 숫자는 Python 숫자, null은 None이 됩니다.
변환한 본문을 Service.run에 전달합니다.

서버의 학점 검증

service.py · integer()와 호출 부분

```
def integer(value, label, minimum, maximum):  
    if type(value) is not int or not minimum <= value <= maximum:  
        raise InputError(  
            f'{label}: {minimum}~{maximum} '  
            '범위의 정수가 필요합니다.')    return value
```

```
credits = integer(body['tot_cred'], '이수 학점', 0, 200)
```

학점 201은 SQL 실행 전에 거부합니다.

화면 검사를 우회한 요청도 서버에서 확인합니다.

서비스가 저장소에 전달하는 값

service.py · Service.run()의 POST 분기

```
sid = integer(body['id'], '학번', 1, 2147483647)
name = text(body['name'], '이름')
dept = body['dept_name']
if dept is not None:
    dept = text(dept, '학과')
return self.repository.add((sid, name, dept, credits))
```

필드 검증을 끝내고 값 튜플을 전달합니다.
튜플 순서는 INSERT 열 순서와 같아야 합니다.

SQLite 연결과 외래키 설정

repository.py · connect()

```
db = sqlite3.connect(self.path, timeout=3)
db.row_factory = sqlite3.Row
db.execute('PRAGMA foreign_keys = ON')
return db
```

외래키 검사는 새 연결마다 겁니다.
sqlite3.Row는 조회 행을 열 이름으로 다루게 합니다.

SQL과 매개변수를 나누어 실행

repository.py · add() 호출 내용 · 줄바꿈 조정

```
return self.execute(
    'INSERT INTO student(id, name, dept_name, tot_cred) '
    'VALUES (?, ?, ?, ?)',
    values, True
)
```

values = (2001, "O'Brien", "컴퓨터공학", 12)
SQL의 네 자리와 튜플의 네 값을 순서대로 연결합니다.

실행 결과와 변경 행 수

repository.py · execute() 발췌

```
with closing(self.connect()) as db, db:
    cursor = db.execute(sql, params)
    rows = [] if write else [
        dict(row) for row in cursor.fetchall()]
    changes = cursor.rowcount if write else 0
```

SELECT는 rows로, DML은 changes로 결과를 확인합니다.
SQL 템플릿과 params를 execute에 별도로 전달합니다.

커밋·롤백·연결 해제

with db

정상 종료는 커밋, 예외 전파는 롤백을 처리합니다.

closing(connection)

마지막에 SQLite 연결을 닫습니다.

오류 전파

DB 예외를 HTTP 처리기까지 전달해 오류 응답을 만듭니다.

연결의 with만 사용해도 연결이 자동으로 닫힌다고 가정하지 않습니다.

UPDATE와 DELETE의 바인딩 순서

repository.py · update(), delete() · 줄바꿈 조정

```
return self.execute(
    'UPDATE student SET tot_cred = ? WHERE id = ?',
    (credits, student_id), True)

return self.execute(
    'DELETE FROM student WHERE id = ?', (student_id,), True)
```

학번 2001을 24학점으로 수정할 params는 (24, 2001)입니다.
원소 하나인 튜플에는 심표가 필요합니다.

없는 학생과 변경 행 수 0

service.py · Service.run()의 공통 처리

```
if not result['changes']:
    raise NotFound('해당 학생이 없습니다.')
return result
```

0행 UPDATE·DELETE는 SQLite 실행 오류가 아닐 수 있습니다.
서비스가 0행을 해석하여 HTTP 404로 바꿉니다.

오류 발생 계층

실험	결과	발생 계층
학번 2001 중복 등록	409 · PK 위반	데이터베이스
API로 학점 201 전송	400 · 범위 검사	애플리케이션
없는 학과로 등록	409 · FK 위반	데이터베이스
없는 학생 수정·삭제	404 · 대상 없음	애플리케이션

실습지 4번의 요청을 실행하고 실패 후 데이터가 유지되는지 확인합니다.

응답의 HTTP 상태와 본문

server.py · dispatch(), send() 발췌

```
result.update(ok=True)
self.send(201 if self.command == 'POST' else 200, result)
```

```
# send() 안에서 JSON 바이트를 만든 뒤 전송
self.send_response(status)
self.end_headers()
self.wfile.write(data)
```

Python dict를 JSON 바이트로 바꾸어 브라우저에 보냅니다.
상태 코드와 응답 본문은 서로 다른 부분입니다.

오류 응답 검사와 화면 갱신

static/app.js · request(), render() 발취

```
if (!response.ok)
  throw new Error(
    `HTTP ${response.status} · ${data.error}`);

// render()는 값으로 HTML을 조립하지 않습니다.
td.textContent = row[key] === null ? '미정 (NULL)' : row[key];
```

fetch는 HTTP 409를 받아도 응답을 반환할 수 있습니다.
textContent는 학생 이름을 HTML 대신 텍스트로 표시합니다.

NULL과 SQL처럼 보이는 이름

NULL

2002 / 가상학생 / 미정 (NULL) / 0을 등록합니다.

문자열

2003의 이름을 x' OR 1=1 -- 로 등록합니다.

검색 결과

같은 이름을 검색하면 2003 학생 한 명만 나옵니다.

요청 JSON의 null과 문자열 "NULL"은 다른 값입니다.

확장 과제: 이름 접두어 검색

repository.py · Repository.list()의 조건과 params 변경

```
# 변경할 SQL 조건
```

```
WHERE substr(name, 1, length(?)) = ?
```

```
# 이름 조건이 있을 때 전달할 값  
(name, name)
```

초기 데이터에서 김을 검색하면 김민지 1명이 나옵니다.
두 개의 ?에 각각 같은 이름을 전달합니다.

코드 설명과 수정 결과 제출

기초 확인

dict 접근, 튜플 순서, self, with를 한 문장씩 설명합니다.

요청 추적

등록 요청의 파일·함수와 입력값을 기록합니다.

확장 과제

접두어 검색 코드 수정은 수업 후 선택 과제로 진행합니다.